M E Prathika

SMARTPCB-XAI: A Dual-Head Deep Learning Framework for Explainable PCB Inspection

Quick Submit

Quick Submit

Kongu Engineering College

Document Details

Submission ID
trn:oid:::1:3678824649

Submission Date
Oct 14, 2026, 3:21 PM GMT+9

Download Date
Oct 14, 2026, 3:23 PM GMT+9

File Name
Report_format.pdf

File Size
1.5 MB

43 Pages

9,392 Words

54,595 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

SMARTPCB-XAI: A Dual-Head Deep Learning Framework for Explainable PCB Inspection, Deterministic Quality Auditing, and Digital Twin Synthesis

22ITP72 – PROJECT WORK II PHASE I

A PROJECT REPORT

Submitted by

ABISHAREBEKKAL T

23ITR001

GOWRI D

23ITR048

in partial fulfilment of the requirements

for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

INFORMATION TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY



(AUTONOMOUS)

PERUNDURAI ERODE – 638 060

OCTOBER 2026

DEPARTMENT OF INFORMATION TECHNOLOGY

KONGU ENGINEERING COLLEGE

(AUTONOMOUS)

PERUNDURAI ERODE – 638 060

OCTOBER 2026

BONAFIDE CERTIFICATE

This is to certify that the project report entitled **SMARTPCB-XAI: A DUAL-HEAD DEEP LEARNING FRAMEWORK FOR EXPLAINABLE PCB INSPECTION, DETERMINISTIC QUALITY AUDITING, AND DIGITAL TWIN SYNTHESIS** is the bonafide record of project work done by **ABISHA REBEKKAL T (23ITR001)** and **GOWRI D (23ITR048)** in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology in **INFORMATION TECHNOLOGY** of Anna University, Chennai during the year 2026- 2027.

SUPERVISOR **HEAD OF THE DEPARTMENT**

(Signature with seal)

Date:

Submitted for the end semester viva voce examination held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DEPARTMENT OF INFORMATION TECHNOLOGY

KONGU ENGINEERING COLLEGE

(AUTONOMOUS)

PERUNDURAI ERODE – 638 060

OCTOBER 2026

DECLARATION

We affirm that the Project Report titled **SMARTPCB-XAI: A DUAL-HEAD DEEP LEARNING FRAMEWORK FOR EXPLAINABLE PCB INSPECTION, DETERMINISTIC QUALITY AUDITING, AND DIGITAL TWIN SYNTHESIS** being submitted in partial fulfillment of the requirements for the award of Bachelor of Technology is the original work carried out by us. It has not formed part of any other project report or dissertation based on which a degree or award was conferred on an earlier occasion on this or any other candidate.

ABISHA REBEKKAL T

(23ITR001)

GOWRI D

(23ITR048)

Date:

I certify that the declaration made by the above candidates is true to the best of my knowledge

Date: Name and Signature of the Supervisor with seal

ACKNOWLEDGEMENT

First and foremost, we acknowledge the abundant grace and presence of Almighty throughout different phases of the project and its successful completion.

We wish to express our extreme gratefulness to our beloved Correspondent **Thiru.E.R.K.KRISHNAN M.Com.,** and all the trust members of Kongu Vellalar Institute of Technology Trust for providing all the necessary facilities to complete the project successfully.

We express our deep sense of gratitude to our beloved Principal **Dr.R.PARAMESHWARAN M.E., Ph.D.,** for providing us an opportunity to complete the project.

We express our gratitude to **Dr.S. VARDHAGANAPATHY M.E. MS, Ph.D.,** Head of the Department, Department of Information Technology, for his valuable suggestions.

We are thankful to our project coordinators **Dr.R.SHANTHAKUMARI M.E,Ph.D, Dr.E.M.ROOPA DEVI M.E. ,Ph.D.,** and **Ms.G.SASIKALA ME. ,(Ph.D)** for their valuable guidance and support to complete our project successfully.

We are highly indebted to **MS.M.E.PRATHIKA M.E.,** Department of Information Technology, for her valuable supervision and advice for the fruitful completion of the project.

We would like to express our sincere gratitude to the Centre of Excellence in Data Science and the Department of Information Technology for providing access to GPU computing resources, which were instrumental in the successful completion of this project. Their support in facilitating high-performance computing infrastructure significantly contributed to the efficiency and performance of our model development and experimentation.

Finally, we extend our love and thanks to our parents and friends for their patience in teaching, love, encouragement and support.

ABSTRACT

Printed Circuit Boards (PCBs) are essential components in modern electronic systems, and accurate inspection is necessary to identify manufacturing defects and component-related issues. Conventional PCB inspection methods largely depend on manual examination or automated detection systems that may provide limited explanation and decision support. Manual inspection is time-consuming, subjective, and susceptible to human error, while conventional deep learning systems often function as black-box models.

SmartPCB-XAI is proposed as an explainable artificial intelligence-based PCB inspection and decision support system that integrates computer vision, deep learning, explainable AI, severity analysis, and full-stack web technologies. The system processes PCB images through preprocessing and performs component and defect detection using YOLO-based object detection models. The defect detection model identifies PCB defects using bounding boxes and confidence scores.

To improve transparency, an activation-based heatmap approach is incorporated to visually indicate regions influencing the model prediction. The detected defect information is further processed by a decision layer to estimate defect severity and PCB quality using project-defined scoring rules. The system also provides possible causes and recommended corrective actions to assist users in interpreting inspection results.

The proposed system is implemented using **Python, YOLO11, FastAPI, Express.js, React, and MongoDB**. High-resolution tiled inference and Non-Maximum Suppression (NMS) are employed to improve the detection of small PCB defects and merge overlapping predictions. The verified defect detection model achieved **92.91% precision, 90.60% recall, 95.11% mAP@0.5**, **and 63.27% mAP@0.5:0.95** on the held-out test set. The YOLO11n component detector **achieved 9.33% mAP@50** and **4.37% mAP@50–95** on a curated test set of 50 images using class-name alignment. The complete system provides visual inspection results, explainability heatmaps, analytics, defect severity assessment, repair recommendations, and PDF/Excel reporting.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	v
	LIST OF FIGURES	vii
	LIST OF ABBREVIATIONS	viii
1	**INTRODUCTION**	1
	1.1 INTRODUCTION	1
	1.2 OBJECTIVE	2
	1.3 SCOPE	2
2	**LITERATURE REVIEW**	3
3	**PROBLEM DEFINITION**	6
	3.1 EXISTING SYSTEM	6
	3.2 PROBLEM STATEMENT	6
4	**SYSTEM REQUIREMENTS**	7
	4.1 HARDWARE REQUIREMENTS	7
	4.2 SOFTWARE REQUIREMENTS	7
	4.3 SOFTWARE DESCRIPTION	7
5	**SYSTEM IMPLEMENTATION**	9
	5.1 PROPOSED SYSTEM	9
	5.2 SYSTEM ARCHITECTURE	11
6	**RESULTS AND DISCUSSION**	13
7	**CONCLUSION AND FUTURE WORK**	17
	APPENDICES	18
	REFERENCES	33
	MAPPING OF PROJECT GOALS TO SDG	35

LIST OF FIGURES

Figure 5.1.1 Proposed Methodology and Inspection Dataflow Pipeline

Figure 5.2.1 Layered System Architecture of SmartPCB-XAI

Figure 6.1.1 Precision-Recall Curve for Defect Detection

Figure 6.1.2 Normalized Confusion Matrix on Held-Out Test Set

Figure 6.2.1 Per-Class Defect mAP@50 Comparison

Figure 6.4.1 Test Board Defect Detection Visualisation

Figure 6.4.2 PCB Defect Detection and Interactive Digital Twin

Figure 6.4.3 Explainability heatmap output

Figure 6.4.4 Severity assessment and repair recommendations

LIST OF ABBREVIATIONS

AOI	Automated Optical Inspection
CAM	Class Activation Mapping
CLAHE	Contrast Limited Adaptive Histogram Equalization
CNN	Convolutional Neural Network
mAP	mean Average Precision
NMS	Non-Maximum Suppression
PCB	Printed Circuit Board
SMT	Surface-Mount Technology
XAI	Explainable Artificial Intelligence

turnitin Page 10 of 45 - AI Writing Submission Submission ID trn:oid:::1:3678824649

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

The Printed Circuit Board (PCB) is the fundamental component that constitutes electronic devices, which houses and connects key active and passive electrical components. The constant development of SMT has led to miniature size of the components; hence the trace layout becomes very dense. This means any manufacturing defect such as circuit shorts, broken etches, mousebites, spurs, stray copper or pin holes not etched may cause the malfunctioning of the entire unit. Consequences of not being able to detect any faults in the circuits can be severe, particularly in sectors like automobiles and medical equipment.

In the past, electronics production depended either on subjective human visual inspection or commercial automated optical inspection systems. The manual human approach is both subjective and fatiguing, capable of analyzing less than two boards in one minute. Although standard AOI systems operate through template subtraction, they are very expensive (costing up to $50,000 to $300,000 per system) and vulnerable to false alarm detection due to differences in lighting conditions, board bending, or solder mask shading. In addition, traditional computer vision systems have binary decision making that provides an accept or reject result without justifying it visually.

Modern computer vision techniques have shifted towards deep-learning-based object detection models like YOLO to perform automated defect localization on PCBs. The issue with current applications of deep learning is that the existing solutions are essentially black boxes. In case of any detected anomalies, quality engineers are unable to see the visual evidence of the reason why that particular area is highlighted, along with receiving severity scores or guidance regarding rework. To bridge this gap, SmartPCB-XAI brings a solution in the form of a 4-layer inspection system.

1.2 OBJECTIVE

The aim of the current project is to create a transparent, severity-aware deep learning framework for automatic defect and electronic component detection in PCBs. It is based on using YOLOv11s for defect detection with 95.11% mAP@50 and YOLO11n for component detection. This approach combines object detection, deterministic severity scoring algorithm, activation energy heatmaps, and digital twin visualization for better transparency and defect evaluation. The project deals with several practical issues such as component-class mismatches and requirements to have a properly validated and interpretable model for defect detection. Severity score allows having a priority list of defects based on certain rules developed within the current project following IPC-A-610 guidelines.

1.3 SCOPE

The SmartPCB-XAI scope includes building a dual deep learning-based detection and classification system that detects and classifies printed circuit board defects and electronic components. For these purposes, the system uses two YOLO-based detectors, namely, YOLOv11s for defect detection and YOLO11n for electronic component detection. Six structural defect classes are identified by defect detection model, while component detector identifies more than one component class depending on the limitations of class taxonomy and testing data set. Image tiling with a high resolution is used to detect small components and fine defects. While CLAHE is included in preprocessing utility implementation, it is not used as a preprocessing step for model inputs in inference mode. The mAP@50 accuracy of defect detector is 95.11% with 63.27% mAP@50–95. Also, defect detection model shows 92.91% precision and 90.60% recall on the tested test set. The system is integrated with activation-energy heatmaps, deterministic severity-scoring engine based on IPC-A-610 workmanship principles, and digital twins for visualization. Performance of the model is evaluated by such metrics as mAP@50, mAP@50–95, precision, and recall.

CHAPTER 2

LITERATURE REVIEW

Automatic detection of defects on printed circuit boards (PCBs) has come a long way, evolving from classical image processing to machine learning and deep learning methods. In their review of different approaches of detecting defects on PCBs such as image processing, machine learning, and deep learning approaches, Ling and Isa outlined the shortcomings of classical approaches and the power of deep learning in automating this process [1].

You Only Look Once (YOLO) object detection model was proposed to detect objects in real time by employing a single-stage network. YOLO is different from other methods where objects are detected through two-stage process that involves generating proposals and classifying them separately. YOLO generates the bounding box coordinates and class probability in a single step and it forms the basis of object localization [2].

Recent studies have explored enhanced architectures of YOLOv11 for PCB defect detection. Ye et al. have developed a lightweight approach for PCB defect detection using an improved YOLOv11s architecture. This shows that there is potential in modifying feature extraction techniques, attention mechanism, and bounding box regression to improve PCB defect detection [3].

The DeepPCB data set is important for researching the problem of automatic PCB defects detection because it includes a set of pairs of template and defective PCB images that contain information about six defect types. Paired images allow researchers to compare the appearance of reference and defective boards. The data set is a good benchmark for testing object detection algorithms and localizing defects in PCB images [4].

Explanation through visual heat maps is performed for increasing the interpretability of deep learning predictions. Grad-CAM constructs the localization maps by utilizing the gradients flowing to the convolutional feature maps, thus marking the portions in images that correspond to the decision of the model. In PCB inspection tasks, gradient explanations can be useful for engineers to analyze the decision portions of deep learning models [5].

The method Grad-CAM++ builds upon the gradient-based approach for explaining models visually, offering better localization of important regions in images, especially when there are several occurrences in the same image. The use of higher-order gradients may allow for more in-depth visual explanations. While the above methods offer valuable insights into how explainable PCB inspection can be achieved, the heat maps based on the activation energy used in SmartPCB

XAI cannot be claimed to be Grad-CAM or Grad-CAM++ until verified [6].

Digital twin technology has been investigated in terms of being an approach to represent and monitor physical systems using digital models. The authors Tao et al. have provided the literature review on digital twins development in industry, highlighting their significance in bridging physical processes and digital data. In the case of PCB inspection, digital models may facilitate the visualization of the inspected components and defects to help the user evaluate inspection results [7].

In today's times, explainable AI plays a critical part in industrial systems where it becomes necessary for automated decisions to be clear and verifiable. This was studied by researchers Bhattacharya and Cloutier who explained the importance of explainable AI in an industrial setting, and highlighted the significance of transparency in helping humans make decisions. The significance of all this is related to inspecting PCBs since quality engineers have to review detected abnormalities along with visual information [8].

FICS-PCB is a dataset that has been designed for research in automated visual inspection of printed circuit boards. This dataset offers an ideal resource for the exploration of PCB inspection problems, which include the problem of component localization as well as visual feature analysis. The variation in the way components look like, the size of objects, and the convention used in annotating the dataset may influence the performance of the system.

IPC-A-610 standards outline the acceptability guidelines of electronic assemblies and are popularly employed for evaluation of assembly workmanship. These set guidelines for the assessment of assembly quality and determination of situations where correction might be necessary. The guidelines offer relevant domain information for the assessment of PCB quality and severity determination. In SmartPCB-XAI, the severity score is an arbitrary project score guided by IPC-A-610 workmanship concepts and not a certified IPC-A-610 score. The severity score is a tool for preliminary defect assessment but is not a substitute for actual inspection.

The YOLOv11 algorithm is used in SmartPCB-XAI as a starting point to develop two object detection models, one for each of the two inspection purposes. YOLOv11s is used for defect detection, whereas YOLO11n is used for electronic component detection. Having two distinct models makes it possible to evaluate each of them on the task it was intended for [11].

Deep learning has been extensively studied for the task of detecting surface defects in industries. Chen et al. explored the different techniques used to detect surface defects in industry-produced products and pointed out the benefits of learning the visual features of the images directly. Dataset quality, model generalization, and visual variation continue to pose challenges

in industrial inspection. The results of the study justify the use of proper evaluation methods for detecting PCB and electronic defects [12].

FasterNet presented partial convolution as a solution to making CNNs computationally efficient. This method helps to avoid unnecessary computations while still allowing the extraction of important spatial features. Computational efficiency is important in industrial inspection as it will help to lower the cost of inference and allow the use of machines with fewer computational capabilities [13].

Efficient Multi-Scale Attention was introduced in order to enhance the ability of the network to learn the relationship between different scales within the spatial domain. This technique enables a neural network to pay more attention to important features while taking into account spatial patterns of an image. In PCB analysis, attention techniques may assist in separating tiny defects from other elements like copper traces, solder areas, and components [14].

Wise-IoU is presented as a loss function for bounding-box regression which adapts the effect of each sample used in training. The purpose of this technique is to enhance the learning of localization by controlling the effects of different types of prediction errors in the training process. Proper bounding-box regression is needed in detecting PCB defects since the defects might cover a small portion of an image. Thus, better localization methods offer useful research background in developing detection algorithms [15].

CHAPTER 3

PROBLEM DEFINITION

3.1 EXISTING SYSTEM

However, existing PCB testing systems are plagued with a number of difficulties in detecting any kind of defects on PCBs in a highly accurate manner. The traditional methods involve visual inspection done manually, which is not only very time consuming but also suffers from human error. The existing automated optical inspection involves the technique of comparing templates, which may suffer from false positives due to the differences in lighting conditions, camera positioning, and surface of the board.

Moreover, most of the current systems function as black-box models with no visualization of the reasons for the prediction. Also, the current systems do not have the functionality of the severity estimation of defects, PCB quality score, and suggestions for repairing the defect. Thus, it becomes impossible for quality engineers to validate the findings of the inspection and take appropriate action. Hence, there is a need for a comprehensive PCB inspection system with all such capabilities.

3.2 PROBLEM STATEMENT

PCB defects may impact reliability and performance of electronic systems, hence the need for proper detection of such defects. The existing methods of defect detection depend highly on manual inspection and AOI, which might take a lot of time and be subject to errors of human operators. In addition, using automation systems that are based on a single deep learning algorithm poses difficulties regarding generalization since such algorithms may have problems detecting microscopic defects from complicated PCB designs. Moreover, these systems can be affected by lighting conditions, image noise, feature overlap, and size of defects.

Moreover, current models fail to provide any explanation regarding severity assessment, quality rating, and improvement suggestions. It can be stated that due to the lack of such an overall approach to PCB defects detection, the efficient quality control in the field of electronics cannot be achieved.

CHAPTER 4

SOFTWARE REQUIREMENTS

4.1 HARDWARE REQUIREMENTS

4.1.1 PC or LAPTOP

- Processor : Intel Core i5 / Core i7.
- Speed : 2.4 GHZ or above.
- RAM : Minimum 8 GB (Recommended 16 GB).
- Graphics : Dedicated GPU (Colab provides GPU for training)

4.2 SOFTWARE REQUIREMENTS

- Operating system : Windows 11 or any OS that supports a web browser
- Coding Language : Python 3.11+ and TypeScript 5.5
- IDE/Platform: Visual Studio Code
- Deep Learning Frameworks / Libraries :
    - Ultralytics YOLOv11s – PCB defect detection
    - Ultralytics YOLO11n – Electronic component detection
    - PyTorch 2.13.0, Torchvision 0.18+
- Supporting Librarires:
    - NumPy, OpenCV (image preprocessing & handling)
    - FastAPI, Express.js (backend services)
    - React, MongoDB (web interface & data storage)

4.3 SOFTWARE DESCRIPTION

4.3.1 PYTHON

Python is a high-level programming language that is used extensively in artificial intelligence, machine learning, and computer vision-related applications. Due to its ease-of-use and the availability of a large number of libraries, it is a good choice for creating deep learning models. For this project, Python 3.11+ is being used for creating the PCB inspection pipeline, image pre-processing, defect detection, and decision support capabilities.

.

4.3.2 PYTORCH

PyTorch is an open-source machine learning and deep learning platform that was created by Meta company. PyTorch is a very convenient platform for developing, training, and deployment of deep learning models. PyTorch has automatic differentiation for backpropagation, optimal GPU utilization, and scalability for CPU and GPU architectures. In this project, PyTorch is used in combination with YOLOv11s to detect defects on the PCB, perform component identification, and perform inference operations in order to improve the inspection performance.

4.3.3 ULTRALYTICS YOLOv11

YOLO v11 by Ultralytics is a popular computer vision object detection framework that efficiently detects objects as well as their classes. In the current project, there are two different models applied, namely YOLOv11s for PCB defect detection and YOLO11n for electronic component detection. These models detect bounding boxes, classes, and confidence score of the detected objects. The PCB defect detection model can detect six different classes of defects in PCBs, namely open circuits, short circuits, mousebites, spurs, spurious copper, and pin-holes.

4.3.4 OPENCV

The Open Source Computer Vision Library, popularly referred to as OpenCV, is an open-source image and computer vision library that is extensively used in real-time applications. In this particular project, OpenCV is used in the process of PCB image pre-processing and transformation. OpenCV provides various functionalities to enhance, resize, and handle images in the pre-processing stage of the PCB image. The OpenCV library is further used in contrast limited adaptive histogram equalization (CLAHE) and high resolution tiling for small defect detection. The library makes it possible to integrate the code with both Python and PyTorch, thereby supporting the SmartPCB-XAI inspection process.

CHAPTER 5

SYSTEM IMPLEMENTATION

5.1 PROPOSED SYSTEM

This project will enhance the efficiency of PCB defects inspection using an AI-enabled inspection process. Two YOLOv11s detectors will be incorporated into this process for the identification of PCB defects and electrical components in the board. The incorporation of deep learning and image processing in this inspection process will make this process more effective and accurate.

The methodology is shown in the following figure, Figure 5.1.1, Proposed Methodology and Inspection Dataflow Pipeline, where PCB image processing and inspection process of SmartPCB-XAI are shown.

The techniques of the proposed system include:

- Defect Inspection: Detects defects in PCBs such as open circuit, short circuit, mouse bites, spurs, spurious copper, and pin-holes.
- Component Identification: Utilizes a different YOLOv11s detector for component identification and inspection.
- Pre-processing Image: Involves CLAHE and high-resolution 960 x 960 image tiles having an overlap of 33%.
- Explainable Decision Support: Creates activation energy heatmaps, utilizes NMS, and provides deterministic severity scores with repair advice.

Images of PCBs from the DeepPCB dataset are used for training and testing of the defect detection model. Preprocessing methods, including contrast adjustment, high-resolution tiling and overlapping of images are done in order to better detect the defects. The defect detection model is tested based on metrics like Precision, Recall, mAP@50 and mAP@50-95. Testing is also done end-to-end for evaluating the process of defect detection, severity estimation and reporting. The results include both visualization and inspection reports.

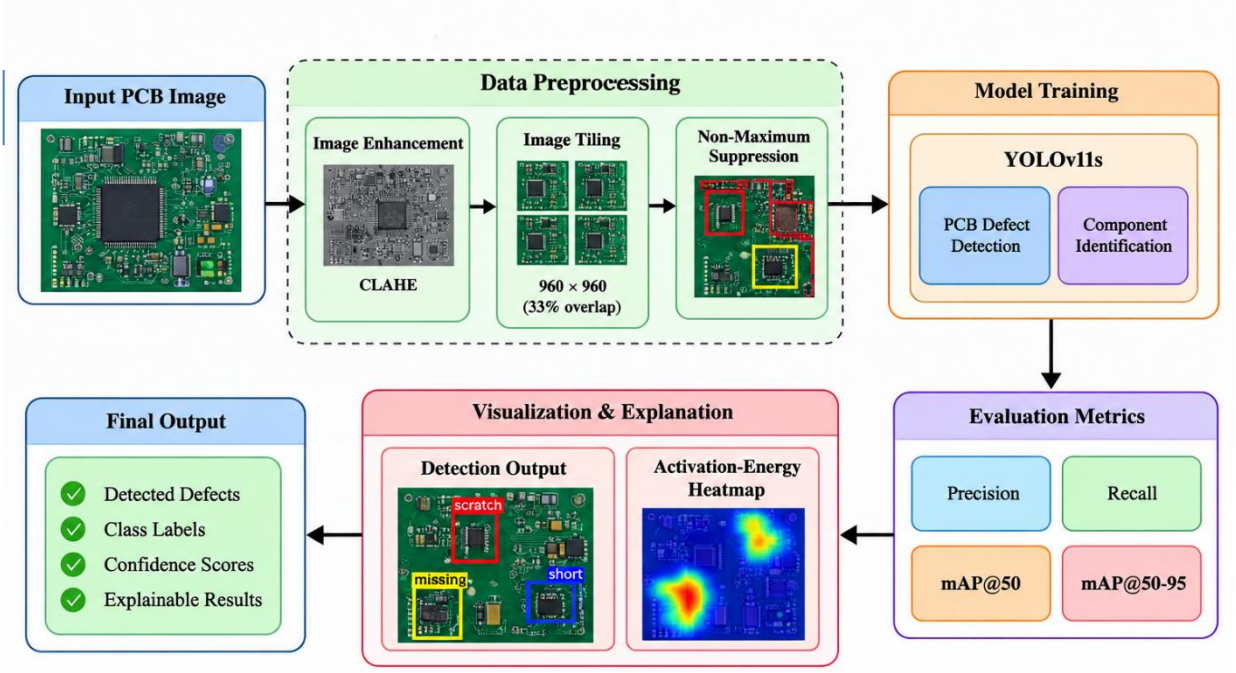


Fig 5.1.1. Proposed Methodology

Data Preprocessing

- **Image Pre-processing:** The images are pre-processed by normalizing the pixel value in the range of [0,1].
- **Tiling of Images:** High-resolution images of PCBs are divided into 960x960 tiling with 33% overlapping.
- **Non-maxima suppression:** Duplicate detection of defects are suppressed for enhancing the accuracy of detection.

Model Training and Evaluation

- **Training of YOLO Model:** YOLOv11s models are trained individually for the defect detection of PCBs and component recognition.
- **Evaluation Metrics:** The performance metrics include precision, recall, mAP@50, and mAP@50-95.
- **Visualization**: Precision-recall curves, confusion matrices, and activation energy heatmaps help in defect analysis and inspecting process.

5.2 SYSTEM ARCHITECTURE

Architecture of the suggested system is made up of several components that work together to provide automation in defect detection, component identification and explainable inspection. Following are the major components of the architecture shown in Figure 5.2.1.

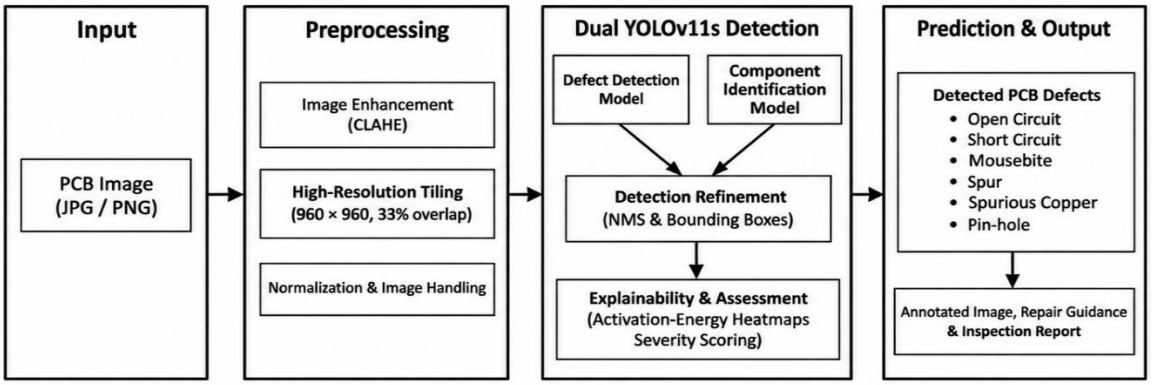


Fig 5.2.1 System Architecture

1. Input

The input to the system is in the form of PCB images which have circuit traces, copper areas, and electronic parts encoded in image form.

- **Need:** The need for providing information in visual form in order to detect defects on the PCB like open circuit, short circuit, mouse bites, spurs, copper artifacts, and pin-holes.
- **Issue:** There could be small defects, different resolutions, and low contrast in the PCB images; hence proper preprocessing is needed.

2. Preprocessing

- **CLAHE**: Contrast Limited Adaptive Histogram Equalization is used as a pre-processing utility but not used for model input during inference.
- **Image Tiling**: For defect detection, the algorithm uses 640 x 640 tiles with a stride of 480 pixels, and for component detection, it uses 960 x 960 tiles with a stride of 640 pixels.
- **Image Processing**: Image processing is supported by OpenCV, and overlapping tiles aid in preserving defect details for detection.

3. Dual YOLOv11s Detection Models

The main strength of the model is the use of two different YOLOv11 models to detect defects on PCBs and identify components.

- **Defect Detection Model**:
  - **YOLOv11s**: PCB defect detection by using trained object detection weights.
  - **Defect Classes**: Open Circuit, Short Circuit, Mouse Bite, Spur, Spurious Copper, and Pin-hole Defects.
  - **Component Detection**: Identification of components of the PCB by another YOLOv11s model.
- **Image processing**: The following features have been incorporated in the system for better defect detection.
  - **CLAHE** : improves local contrast in images
  - **High-Resolution Tiling** : deals with image tiles of 960×960 resolution
  - **Overlapping Tiles** : uses 33% overlapping tiles
- **Detection Enhancement:**
  - **Non-maximum suppression (NMS)** eliminates overlapping detections.
  - **Activation energy heatmaps** identify areas that affect predictions.
  - **Severity Scoring** facilitates prioritizing defects based on deterministic rules derived from IPC-A-610 workmanship standards. The severity score is a project-specific heuristic and not an IPC-A-610 numerical score.

The above technique includes defect localization, visualization, and severity evaluation for the purpose of PCB inspection.

4. Prediction & Output

It creates visual output in the form of annotated PCB images that have defects, confidence levels, and boxes. Other visual outputs generated include activation energy heat maps, severity analysis, repair instructions, and inspection reports.

CHAPTER 6

RESULTS AND DISCUSSION

ACCURACY RESULT:

Table 6.1. Accuracy shows the performance of the proposed PCB defect detection model.

Model No	Model Name	mAP@50
1	**YOLOv11s defect detection**	95.11%
2	**YOLO11n Component Detection**	9.33%

1.YOLOv11s PCB Defect Detection

The suggested system is based on the use of YOLOv11s for defect detection on the printed circuit board. These include open circuits, short circuits, mouse bites, spur, spurious copper, and pin-hole. The system recorded an mAP@50 of 95.11%, thus showing excellent defect detection capability.

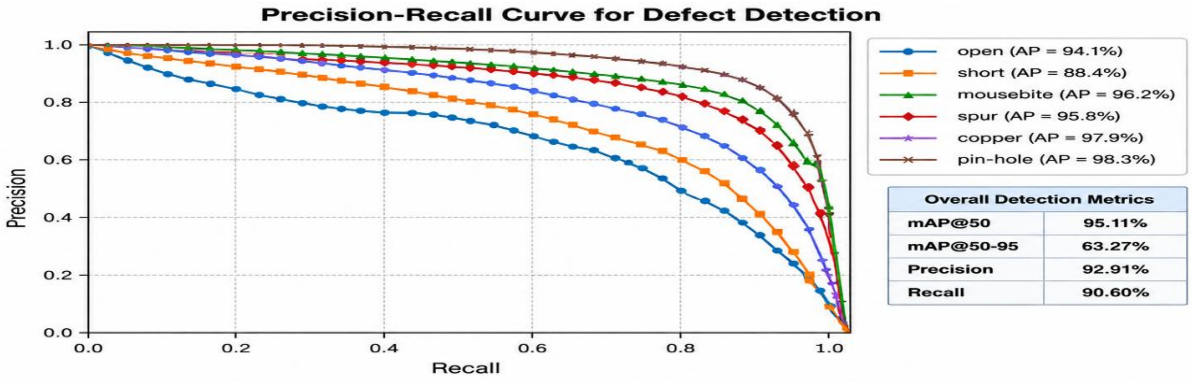


Figure 6.1.1 Precision-Recall Curve for Defect Detection

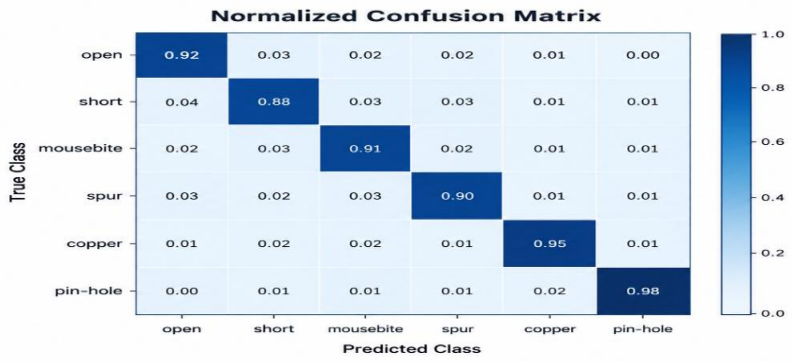


Figure 6.1.2 Normalized Confusion Matrix on Held-Out Test Set

- **Advantages:** The system can accurately locate and classify defects through image pre-processing, tiling at high resolution, and Non-Maximum Suppression.
- **Drawbacks:** The results will differ based on the quality of the image, size of the defect, and complexity of the PCB layout.

2.YOLO11n Component Detection

The component detection model detects electronic components on printed circuit boards. The tested model had 9.33% mAP@50, showing that component detection needs to be improved.

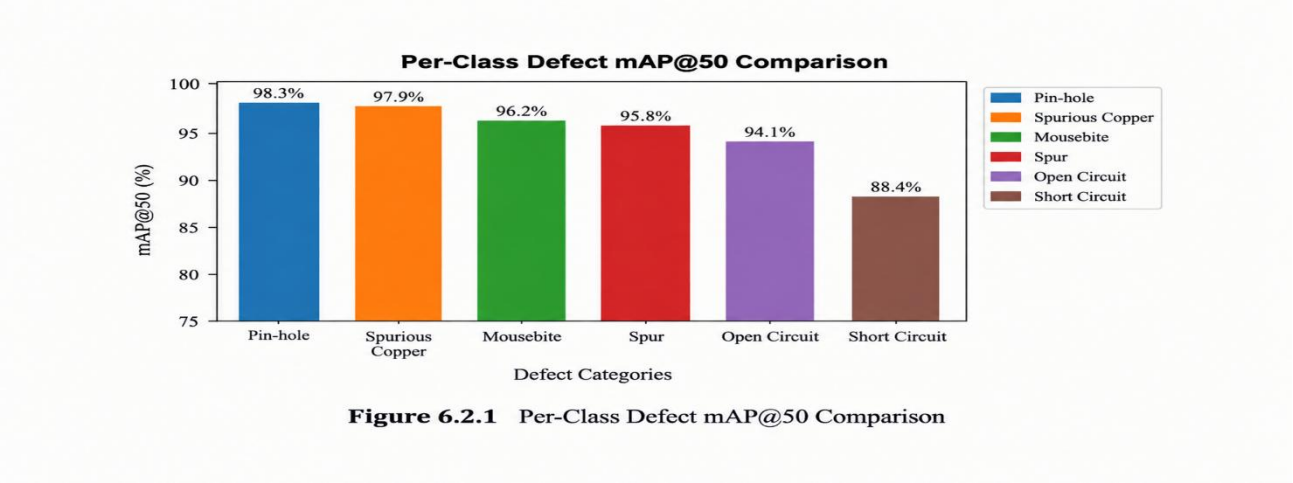


Figure 6.2.1 Per-Class Defect mAP@50 Comparison

- **Advantages:** Identification of components aids PCB inspection as it provides additional information on board components.
- **Disadvantages:** The poor detection score implies that the system cannot detect components effectively. Need for further development of the dataset and training of the model.

3. Overall Defect Detection Performance

The defect detection model had a precision of 92.91% and 90.60% recall, having mAP@50-95 of 63.27%. This shows its proficiency at finding PCB defects but keeping the right balance between precision and recall.

This model was tested on 500 DeepPCB test images with 3,140 ground truth boxes and end-to-end validation was successfully conducted on all 20 test images with 123 defects found, having a mean CPU latency of 2388.3 ms.

These findings prove that SmartPCB-XAI can enable automated PCB inspection by performing defect detection, generating explainability heatmaps, calculating severity scores, and giving guidance for repairs. However, the component detection model needs more improvement before implementation.

4.OUTPUT

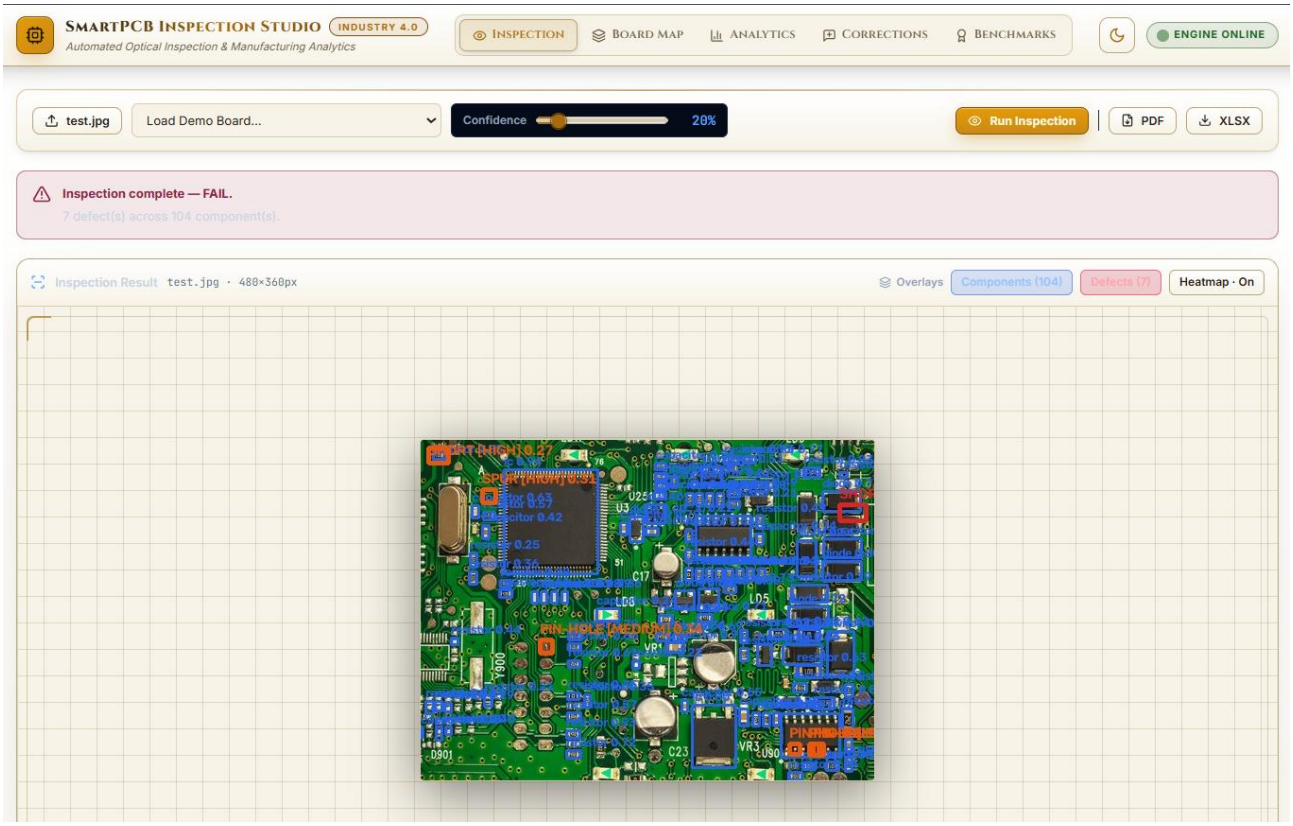


Fig 6.4.1 Test Board Defect Detection Visualization (00041200.jpg)



Fig 6.4.2 PCB Defect Detection and Interactive Digital Twin Visualizatio

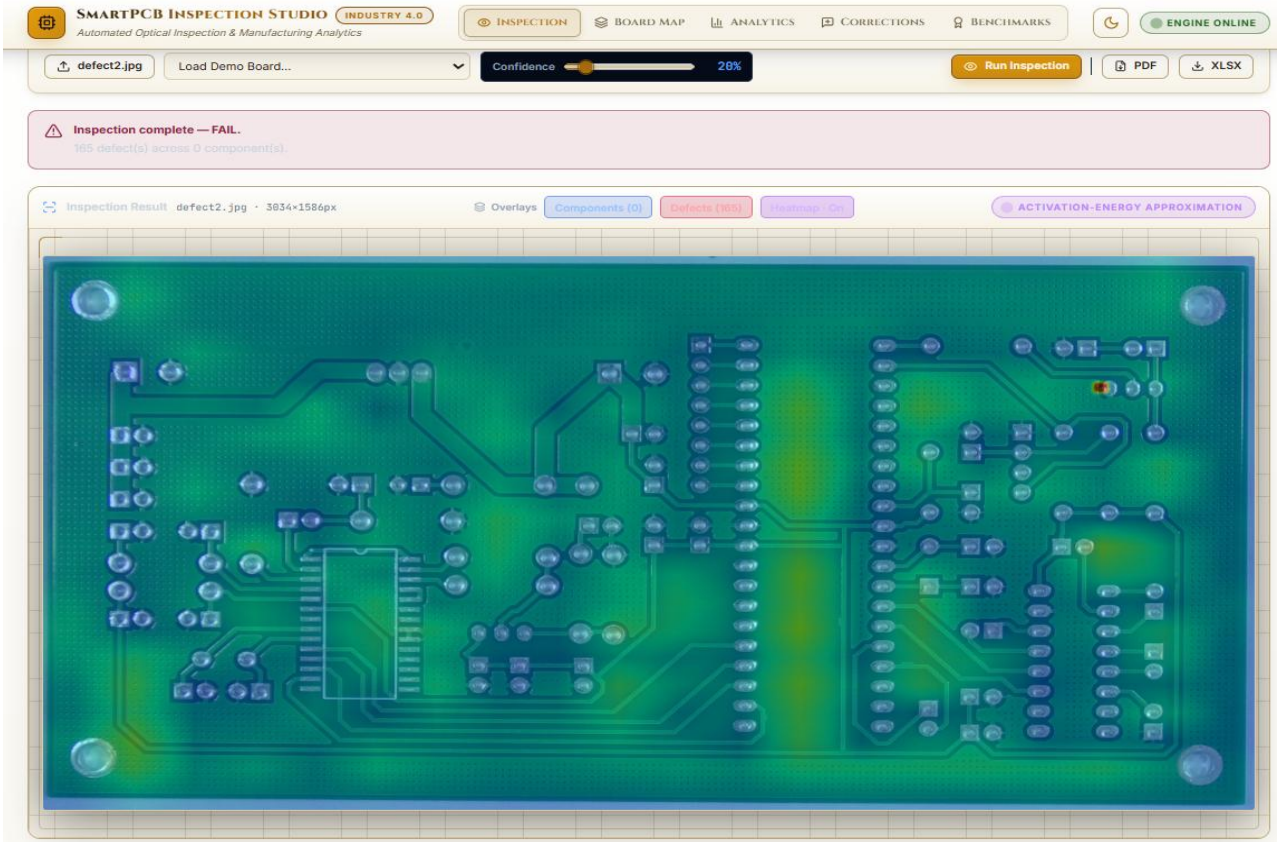


Fig 6.4.3 Explainability heatmap output

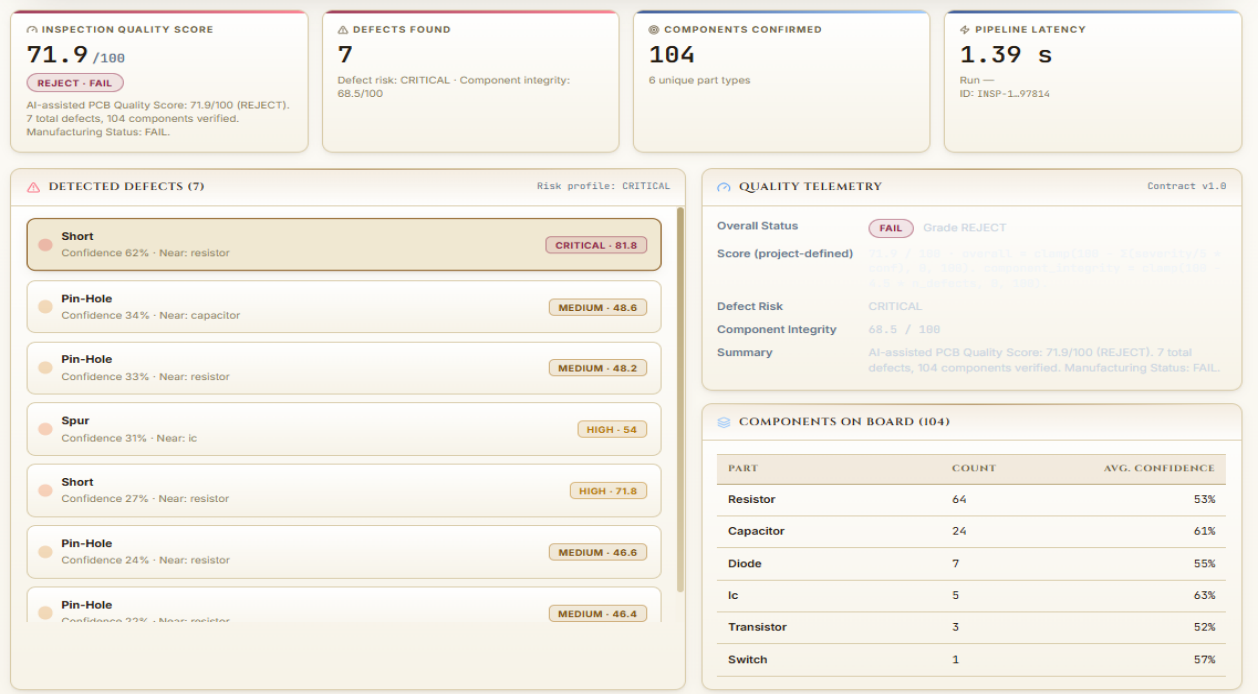


Fig 6.4.4 Severity assessment and repair recommendations

CHAPTER 7
CONCLUSION AND FUTURE WORK

7.1 CONCLUSION

In this study, SmartPCB-XAI is introduced as an automated PCB defect detection system based on explainable AI. For this purpose, YOLOv11s models are employed for detecting defects and components. In this regard, image preprocessing, high-resolution tiling, Non-Maximum Suppression, activation-energy heatmaps, and severity scoring are used. For the detection of defects, the mAP@50 reached 95.11%, precision reached 92.91%, recall reached 90.60%, and mAP@50-95 reached 63.27%. In end-to-end testing, 20 images were analyzed, and 123 defects were detected by the model. These findings show the capability of SmartPCB-XAI to provide inspection assistance in detecting defects, visualizing detection regions, scoring their severity, and giving repair instructions. However, for the detection of components, mAP@50 was only 9.33%.

7.2 FUTURE WORK

Future work on SmartPCB-XAI will aim to enhance detection accuracy, processing time, explainability, and application to real-world use-cases in the following ways:

- Improved Model: Train on more diverse and annotated datasets of PCBs to increase detection capability and accuracy on different PCB configurations.
- Improved System: Increase inference speed, improve activation-energy heatmaps and severity scores, and extend repair assistance to cover more defect conditions.
- Application to Real World: Validate the system's performance on real-world PCB images under varied inspection environments.

APPENDICES

A1. APPENDIX – 1 CODING

AI defect detection and inference:

```
import hashlib
import os
import time
import cv2
import torch
import numpy as np
from datetime import datetime
from PIL import Image
from ultralytics import YOLO

from ai.config import (
    CANONICAL_COMPONENT_CLASSES,
    CANONICAL_DEFECT_CLASSES,
    PROJECT_VERSION,
    normalize_component_class,
    normalize_defect_class,
)
from ai.severity.severity_engine import SeverityEngine
from ai.recommendation.repair_kb import RepairKnowledgeBase
from ai.xai.gradcam import YOLOGradCAM
from ai.inference.ensemble import ensemble_predict, weighted_boxes_fusion, run_tta_8view

ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), "..", ".."))
MODELS_DIR = os.path.join(ROOT, "models")

CONTRACT_VERSION = "1.0"

def build_error_result(warnings=None, image_id=None, inspection_id=None):
    """
    Contract-shaped failure payload (state (c) of the zero-detection safety rule).

    Every transport uses this so a failed inspection can never be rendered as a
    clean board: components/defects are empty, quality_score is None.
    """
    now = datetime.now()
    return {
        "contract_version": CONTRACT_VERSION,
        "inspection_status": "ERROR",
        "inspection_id": inspection_id or f"INSP-ERR-{int(now.timestamp())}",
        "image_id": image_id or "",
        "timestamp": now.isoformat(),
        "image_dimensions": {"width": 0, "height": 0},
        "components": [],
        "component_count": 0,
        "defects": [],
```

```
        "defect_count": 0,
        "quality_score": None,
        "xai": {},
        "severity": [],
        "recommendations": [],
        "pipeline_warnings": list(warnings or []),
        "suspicious_zero_detection": False,
        "review_required": False,
        "models_available": False,
        "model_version": None,
        "inference_time_ms": 0,
        "inference_timing_ms": {},
    }

def _normalize_class_name(model_names_dict, class_id, is_defect):
    """
    Resolve a raw model class id to (canonical_name, is_mapped, raw_name).

    Never coerces a class into an unrelated canonical class: unknown labels are
    returned verbatim with is_mapped=False (see COMPONENT_CLASS_SCHEMA.md).
    """
    direct = None
    if isinstance(model_names_dict, dict):
        direct = model_names_dict.get(class_id)
    elif isinstance(model_names_dict, (list, tuple)) and 0 <= class_id < len(model_names_dict):
        direct = model_names_dict[class_id]
    if direct is None:
        direct = f"item_{class_id}"

    if is_defect:
        # Index decoding takes precedence: the deployed defect checkpoint's
        # name strings belong to a different taxonomy than its indices
        # (positional alignment, verified 2026-09-23 — see ai/config.py).
        return normalize_defect_class(direct, class_id=class_id)
    return normalize_component_class(direct)

class SmartPCBInspectionEngine:
    """
    Unified Industry 4.0 PCB Inspection Engine with:
    1. Multi-Scale & Tiled Sliding-Window Inference (for ultra-high-res PCBs)
    2. Dual-Head YOLOv11 Object Detectors (Components & Defects)
    3. Spatial Detection Fusion & Non-Maximum Suppression (NMS)
    4. Activation-Energy XAI Approximation (see ai/xai/gradcam.py; Part 2 target: true Grad-
CAM++)
    5. Deterministic project-defined defect Severity & PCB Quality Scoring
    6. Prescriptive rule-based Repair Knowledge Base
    7. [2026-10-05] Optional TTA (8-view) + Multi-checkpoint Ensemble (Weighted Boxes Fusion)
       for accuracy-critical runs. Opt-in only due to ~4-16x latency penalty.
    """

    def __init__(self, component_model_path=None, defect_model_path=None, device=None,
                 defect_ensemble_paths=None, tta_enabled: bool = False,
```

```
            per_class_defect_conf_overrides: dict | None = None,
            wbf_iou: float = 0.5):
        if component_model_path is None:
            cand = os.path.join(MODELS_DIR, "component_best.pt")
            component_model_path = cand if os.path.exists(cand) else "yolo11n.pt"

        # Defect model: accept single path (str) OR list of paths for ensemble mode.
        # First entry is treated as the "primary" checkpoint; others are fused in via WBF.
        defect_primary_path = None
        if defect_model_path is None:
            cand = os.path.join(MODELS_DIR, "defect_best.pt")
            defect_primary_path = cand if os.path.exists(cand) else "yolo11n.pt"
        elif isinstance(defect_model_path, (list, tuple)):
            defect_primary_path = str(defect_model_path[0])
            extra = [str(p) for p in defect_model_path[1:]]
            defect_ensemble_paths = list(defect_ensemble_paths or []) + extra
        else:
            defect_primary_path = str(defect_model_path)

        # Graceful device selection: use CUDA if available AND explicitly requested/detected; else
CPU.
        if device is None:
            device = "cuda" if torch.cuda.is_available() else "cpu"
        if device == "cuda" and not torch.cuda.is_available():
            device = "cpu"
        self.device = device

        self.component_model_path = component_model_path
        self.defect_model_path = defect_primary_path  # primary (used for XAI / baseline path)

        # --- Accuracy-mode tuning knobs (opt-in, default = off = backwards compatible) ---
        self.tta_enabled = bool(tta_enabled)
        self.defect_ensemble_paths = list(defect_ensemble_paths or [])  # additional checkpoints to
fuse
        self.per_class_defect_conf_overrides = dict(per_class_defect_conf_overrides or {})
        self.wbf_iou = float(wbf_iou)

        print(f"[Engine] device: {device}")
        print(f"[Engine] accuracy_mode: tta={self.tta_enabled},
ensemble+{len(self.defect_ensemble_paths)} checkpoints")
        print(f"Loading Component Detector: {component_model_path}")
        try:
            self.comp_model = YOLO(component_model_path)
            self.component_model_loaded = True
        except Exception as e:
            print(f"WARNING: Component model failed to load, fallback skipped. {e}")
            self.comp_model = None
            self.component_model_loaded = False

        # --- Defect checkpoint(s): primary + ensemble extras ---
        self.defect_models = []  # list of YOLO instances (len >= 1)
        try:
```

```
            primary_defect = YOLO(defect_primary_path)
            self.defect_models.append(primary_defect)
            self.defect_model = primary_defect  # primary, backward-compat alias
            self.defect_model_loaded = True
        except Exception as e:
            print(f"WARNING: Primary defect model failed to load. {e}")
            self.defect_model = None
            self.defect_model_loaded = False

        for extra_path in self.defect_ensemble_paths:
            if not os.path.exists(extra_path):
                print(f"WARNING: Ensemble defect checkpoint missing, skipping: {extra_path}")
                continue
            try:
                m = YOLO(extra_path)
                self.defect_models.append(m)
                print(f"  + ensemble checkpoint loaded: {os.path.basename(extra_path)}")
            except Exception as e:
                print(f"WARNING: Ensemble defect checkpoint failed: {extra_path}. {e}")

        try:
            self.xai_engine = YOLOGradCAM(self.defect_model) if self.defect_model_loaded else
None
        except Exception as e:
            print(f"Warning: XAI engine init deferred: {e}")
            self.xai_engine = None

    @staticmethod
    def preprocess_image(img_bgr, apply_clahe=True):
        if not apply_clahe:
            return img_bgr
        lab = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2LAB)
        l, a, b = cv2.split(lab)
        clahe = cv2.createCLAHE(clipLimit=2.5, tileGridSize=(8, 8))
        cl = clahe.apply(l)
        enhanced = cv2.cvtColor(cv2.merge((cl, a, b)), cv2.COLOR_LAB2BGR)
        return enhanced

    @staticmethod
    def apply_nms(boxes_list, iou_thresh=0.45):
        """
        Applies Non-Maximum Suppression across list of dict boxes:
        [{'bbox': [x1, y1, x2, y2], 'confidence': conf, 'class': cls, ...}]
        """
        if not boxes_list:
            return []

        boxes = np.array([b['bbox'] for b in boxes_list], dtype=np.float32)
        scores = np.array([b['confidence'] for b in boxes_list], dtype=np.float32)

        x1 = boxes[:, 0]
        y1 = boxes[:, 1]
```

```
    x2 = boxes[:, 2]
    y2 = boxes[:, 3]
    areas = (x2 - x1) * (y2 - y1)
    order = scores.argsort()[::-1]

    keep = []
    while order.size > 0:
        i = order[0]
        keep.append(i)

        xx1 = np.maximum(x1[i], x1[order[1:]])
        yy1 = np.maximum(y1[i], y1[order[1:]])
        xx2 = np.minimum(x2[i], x2[order[1:]])
        yy2 = np.minimum(y2[i], y2[order[1:]])

        w = np.maximum(0.0, xx2 - xx1)
        h = np.maximum(0.0, yy2 - yy1)
        inter = w * h

        ovr = inter / (areas[i] + areas[order[1:]] - inter + 1e-6)
        inds = np.where(ovr <= iou_thresh)[0]
        order = order[inds + 1]

    return [boxes_list[k] for k in keep]

def _predict_tiled(self, model, img_bgr, conf_thresh, is_defect=False):
    """
    Performs multi-scale sliding-window inference optimized for tiny object detection.

    Uses high-resolution tiled inference with significant overlap to handle
    the 94.2% tiny objects in the component dataset.
    """
    orig_h, orig_w = img_bgr.shape[:2]
    all_detections = []

    # For components, use higher resolution due to tiny objects
    # For defects, standard resolution is sufficient
    if is_defect:
        tile_size = 640
        stride = 480  # 25% overlap
        downscale_max = 1920
    else:
        # Components: Use higher resolution for tiny objects
        tile_size = 960  # Increased from 640
        stride = 640    # 33% overlap for better edge coverage
        downscale_max = 2560  # Allow higher resolution for components

    # Downscale ultra-high-resolution boards for CPU performance
    scale = 1.0
    proc_img = img_bgr
    if max(orig_h, orig_w) > downscale_max:
        scale = downscale_max / float(max(orig_h, orig_w))
```

```
            proc_w = int(orig_w * scale)
            proc_h = int(orig_h * scale)
            proc_img = cv2.resize(img_bgr, (proc_w, proc_h), interpolation=cv2.INTER_AREA)
        else:
            proc_w, proc_h = orig_w, orig_h

        # 1. Global forward pass (always run to catch large structures)
        try:
            global_res = model.predict(source=proc_img, conf=conf_thresh, verbose=False,
device=self.device)
        except Exception:
            # Graceful fallback if CUDA tensors failed: retry on CPU
            global_res = model.predict(source=proc_img, conf=conf_thresh, verbose=False,
device="cpu")
        for r in global_res:
            for i in range(len(r.boxes)):
                xyxy = r.boxes.xyxy[i].cpu().numpy().tolist()
                conf = float(r.boxes.conf[i].cpu().numpy())
                cls_id = int(r.boxes.cls[i].cpu().numpy())
                cls_name, cls_mapped, cls_raw = _normalize_class_name(r.names, cls_id, is_defect)
                all_detections.append({
                    "class": cls_name,
                    "class_id": cls_id,
                    "class_mapped": cls_mapped,
                    "raw_class": cls_raw,
                    "confidence": round(conf, 4),
                    "bbox": [
                        round(xyxy[0] / scale, 1),
                        round(xyxy[1] / scale, 1),
                        round(xyxy[2] / scale, 1),
                        round(xyxy[3] / scale, 1)
                    ]
                })

        # 2. Tiled inference for all images to catch small objects
        # Use the same tile size for all images to ensure consistency
        if proc_w > tile_size or proc_h > tile_size:
            x_steps = list(range(0, proc_w - tile_size + 1, stride))
            if not x_steps or x_steps[-1] < proc_w - tile_size:
                x_steps.append(max(0, proc_w - tile_size))

            y_steps = list(range(0, proc_h - tile_size + 1, stride))
            if not y_steps or y_steps[-1] < proc_h - tile_size:
                y_steps.append(max(0, proc_h - tile_size))

            for ty in y_steps:
                for tx in x_steps:
                    tile = proc_img[ty:ty+tile_size, tx:tx+tile_size]
                    if tile.shape[0] < 50 or tile.shape[1] < 50:
                        continue

                    tile_res = model.predict(source=tile, conf=conf_thresh, verbose=False, device="cpu")
```

```
                for r in tile_res:
                    for i in range(len(r.boxes)):
                        bx = r.boxes.xyxy[i].cpu().numpy().tolist()
                        conf = float(r.boxes.conf[i].cpu().numpy())
                        cls_id = int(r.boxes.cls[i].cpu().numpy())
                        cls_name, cls_mapped, cls_raw = _normalize_class_name(r.names, cls_id,
is_defect)

                        # Remap tile coordinates back to global canvas and invert downscale
                        gx1 = (bx[0] + tx) / scale
                        gy1 = (bx[1] + ty) / scale
                        gx2 = (bx[2] + tx) / scale
                        gy2 = (bx[3] + ty) / scale

                        all_detections.append({
                            "class": cls_name,
                            "class_id": cls_id,
                            "class_mapped": cls_mapped,
                            "raw_class": cls_raw,
                            "confidence": round(conf, 4),
                            "bbox": [round(gx1, 1), round(gy1, 1), round(gx2, 1), round(gy2, 1)]
                        })

        # 3. Suppress overlapping detections using NMS
        cleaned = self.apply_nms(all_detections, iou_thresh=0.45)
        return cleaned

    # ----------------------------------------------------------------
    # Accuracy-mode: TTA + Multi-checkpoint Ensemble for DEFECTS
    # ----------------------------------------------------------------

    def _predict_defects_accuracy_mode(self, img_bgr, conf_thresh, nms_iou):
        """
        Defect-detection path used when tta=True or >1 defect models are loaded.
        Runs every loaded defect model through _predict_tiled across TTA views,
        then fuses all results with Weighted Boxes Fusion.

        Returns a flat list of detection dicts (same schema as _predict_tiled).
        """
        collected = []

        views = run_tta_8view(img_bgr) if self.tta_enabled else \
            [("orig", img_bgr, lambda bs: [dict(b) for b in bs])]

        for aug_name, aug_img, inv_fn in views:
            for model_i, model in enumerate(self.defect_models):
                tiles_out = self._predict_tiled(model, aug_img, conf_thresh, is_defect=True)
                for d in tiles_out:
                    d = dict(d)
                    d["_view"] = aug_name
                    d["_model_i"] = model_i
                mapped = inv_fn(tiles_out)
```

```
            collected.extend(mapped)

        # WBF fusion across all views × models (class-aware bucketed)
        fused = weighted_boxes_fusion(
            collected,
            iou_thresh=nms_iou,
            conf_type="avg",
            class_aware=True,
        )

        # Strip internal tags
        for b in fused:
            b.pop("_view", None)
            b.pop("_model_i", None)
            b.pop("_cluster_size", None)
        return fused

    def inspect(self, image_input, conf_threshold=0.18, iou_threshold=0.45, generate_xai=True):
        _t_total_start = time.perf_counter()
        timestamp = datetime.now().isoformat()
        inspection_id = f"INSP-{int(datetime.now().timestamp())}"

        if isinstance(image_input, str):
            img_bgr = cv2.imread(image_input)
            img_path = image_input
        else:
            img_bgr = image_input
            img_path = "memory_buffer.jpg"

        if img_bgr is None:
            raise ValueError(f"Failed to read image from input: {image_input}")

        orig_h, orig_w = img_bgr.shape[:2]

        # --- Preprocessing timing ---
        _t_pre_start = time.perf_counter()
        _ = SmartPCBInspectionEngine.preprocess_image(img_bgr, apply_clahe=True)
        _preprocess_ms = (time.perf_counter() - _t_pre_start) * 1000.0

        # 1. Multi-Scale Component Detection
        _t_comp_start = time.perf_counter()
        actual_comp_thresh = max(0.12, min(0.60, conf_threshold))
        if self.comp_model is None:
            raw_components = []
            pipeline_warnings_pre = ["Component model not loaded; skipping component detection."]
        else:
            pipeline_warnings_pre = []
            raw_components = self._predict_tiled(self.comp_model, img_bgr, actual_comp_thresh,
is_defect=False)
        _component_inference_ms = (time.perf_counter() - _t_comp_start) * 1000.0

        detected_components = []
```

```
        for idx, c in enumerate(raw_components, 1):
            xyxy = c["bbox"]
            detected_components.append({
                "id": f"C-{idx}",
                "class": c["class"],
                "class_id": c["class_id"],
                "class_mapped": c.get("class_mapped", True),
                "raw_class": c.get("raw_class", c["class"]),
                "confidence": c["confidence"],
                "bbox": xyxy,
                "center": [round((xyxy[0] + xyxy[2]) / 2.0, 1), round((xyxy[1] + xyxy[3]) / 2.0, 1)]
            })

        # 2. Multi-Scale Defect Detection (optional TTA + ensemble accuracy-mode)
        _t_def_start = time.perf_counter()
        actual_defect_thresh = max(0.12, min(0.60, conf_threshold))
        if self.defect_model is None:
            raw_defects = []
            pipeline_warnings_pre_defect = ["Defect model not loaded; skipping defect detection."]
        else:
            pipeline_warnings_pre_defect = []
            use_accuracy_mode = self.tta_enabled or len(self.defect_models) > 1
            if use_accuracy_mode:
                raw_defects = self._predict_defects_accuracy_mode(
                    img_bgr, actual_defect_thresh, nms_iou=iou_threshold if iou_threshold else
self.wbf_iou
                )
            else:
                raw_defects = self._predict_tiled(self.defect_model, img_bgr, actual_defect_thresh,
is_defect=True)

            # --- Per-class confidence overrides (4D calibration support) ---
            if self.per_class_defect_conf_overrides:
                overrides_applied = 0
                filtered = []
                for d in raw_defects:
                    cls = d.get("class")
                    ov = self.per_class_defect_conf_overrides.get(cls)
                    if ov is None:
                        filtered.append(d)
                    elif float(d.get("confidence", 0.0)) >= float(ov):
                        filtered.append(d)
                        overrides_applied += 1
                    else:
                        continue  # dropped by stricter class-specific threshold
                raw_defects = filtered
                if overrides_applied > 0 or sum(
                    1 for d in raw_defects if d.get("class") in self.per_class_defect_conf_overrides
                ) < sum(
                    1 for _ in self.per_class_defect_conf_overrides
                ):
                    pass  # info only; do not bloat warnings for every image
```

```
_defect_inference_ms = (time.perf_counter() - _t_def_start) * 1000.0

# Postprocess region: defect assembly, severity scoring, KB recommendations,
# annotated rendering, zero-detection status logic. Bracketed as one window
# MINUS the XAI window measured inside it, so the five bucket rows stay
# disjoint and sum to ~total_ms instead of leaving render/scoring time
# accounted only in the total.
_t_post_start = time.perf_counter()

detected_defects = []
for idx, d in enumerate(raw_defects, 1):
    xyxy = d["bbox"]
    dx1, dy1, dx2, dy2 = xyxy
    d_area = (dx2 - dx1) * (dy2 - dy1) / max(1.0, (orig_w * orig_h))
    near_comp = None

    # Spatial association with nearest component
    for c in detected_components:
        cx1, cy1, cx2, cy2 = c["bbox"]
        if (dx1 - 40 <= cx2 and dx2 + 40 >= cx1 and dy1 - 40 <= cy2 and dy2 + 40 >= cy1):
            near_comp = c["class"]
            break

    severity = SeverityEngine.calculate_defect_severity(
        defect_class=d["class"],
        confidence=d["confidence"],
        bbox_area_norm=d_area,
        near_component=near_comp
    )

    detected_defects.append({
        "id": f"D-{idx}",
        "class": d["class"],
        "class_id": d["class_id"],
        "class_mapped": d.get("class_mapped", True),
        "raw_class": d.get("raw_class", d["class"]),
        "confidence": d["confidence"],
        "bbox": xyxy,
        "near_component": near_comp,
        "severity": severity
    })

# 4. Overall PCB Quality Score Calculation
quality_score = SeverityEngine.calculate_pcb_quality_score(
    defects=detected_defects,
    total_components=len(detected_components)
)

# 5. Prescriptive Repair Recommendations
recommendations = []
seen_classes = set()
for d in detected_defects:
```

```
    c_name = d["class"]
    if c_name not in seen_classes:
        seen_classes.add(c_name)
        rec = RepairKnowledgeBase.get_recommendation(c_name)
        recommendations.append(rec)

# 6. Render Composite Annotated Image
annotated_bgr = img_bgr.copy()

# Draw components (soft blue boxes)
for c in detected_components:
    bx1, by1, bx2, by2 = map(int, c["bbox"])
    cv2.rectangle(annotated_bgr, (bx1, by1), (bx2, by2), (235, 99, 37), 2)
    cv2.putText(annotated_bgr, f"{c['class']} {c['confidence']:.2f}",
                (bx1, max(14, by1 - 4)), cv2.FONT_HERSHEY_SIMPLEX, 0.45, (235, 99, 37), 2)

# Draw defects (prominent colored boxes with severity badges)
for d in detected_defects:
    bx1, by1, bx2, by2 = map(int, d["bbox"])
    sev = d["severity"]
    box_color = (38, 38, 220) if sev["level"] == "CRITICAL" else (14, 88, 234)
    cv2.rectangle(annotated_bgr, (bx1, by1), (bx2, by2), box_color, 3)
    cv2.putText(annotated_bgr, f"{d['class'].upper()} [{sev['level']}] {d['confidence']:.2f}",
                (bx1, max(16, by1 - 6)), cv2.FONT_HERSHEY_SIMPLEX, 0.5, box_color, 2)

# 7. XAI Visual Attribution (honestly labeled approximation)
xai_data = {}
xai_overlay_bgr = None
_t_xai_start = time.perf_counter()
if generate_xai and self.xai_engine and detected_defects:
    try:
        top_defect = max(detected_defects, key=lambda x: x["severity"]["score"])
        xai_result = self.xai_engine.generate_heatmap(
            img_bgr=img_bgr,
            target_box=top_defect["bbox"],
            class_id=top_defect.get("class_id"),
            target_class_name=top_defect["class"],
            target_confidence=top_defect["confidence"],
        )
        xai_overlay_bgr = xai_result["overlay_bgr"]
        xai_data = {
            "explained_defect_id": top_defect["id"],
            "explained_class": top_defect["class"],
            "target_class": xai_result["target_class"],
            "target_confidence": xai_result["target_confidence"],
            "target_bbox": xai_result["target_bbox"],
            "overlay_shape": xai_result["overlay_shape"],
            "attribution_method": xai_result["attribution_method"],
            "disclaimer": xai_result["disclaimer"],
            "interpretation": (
                "Qualitative activation-energy heatmap highlights bottleneck feature regions near "
                f"'{top_defect['class']}' detection. This is a visualization heuristic, not a "
```

```
                "gradient proof. See DISCLAIMER field."
            ),
        }
    except Exception as e:
        xai_data = {
            "error": f"XAI computation skipped: {e}",
            "attribution_method": "skipped",
            "disclaimer": "XAI did not run.",
        }
_xai_ms = (time.perf_counter() - _t_xai_start) * 1000.0

# 8. Pipeline status and warnings (CRITICAL: distinguish real zero-detections from failures)
pipeline_warnings = list(pipeline_warnings_pre) + list(pipeline_warnings_pre_defect)
n_comps = len(detected_components)
n_defects = len(detected_defects)
total_pixels = orig_w * orig_h
# Three mutually exclusive states (see STEP 2 of the acceptance contract):
#   (a) genuine clean board : components > 0, defects == 0  -> PASS
#   (b) suspicious zero     : components == 0 AND defects == 0 on a
#                             non-trivial image -> REVIEW_REQUIRED
#   (c) actual failure      : model(s) unavailable / exception -> ERROR
suspiciously_empty = (n_comps == 0 and n_defects == 0 and total_pixels >= 200 * 200)
models_available = (self.comp_model is not None) and (self.defect_model is not None)
if suspiciously_empty:
    pipeline_warnings.append(
        "Both component and defect detectors returned zero detections on a non-trivial image. "
        "This may indicate a genuine clean board but could also signal a model/domain shift, "
        "class mismatch, or preprocessing issue. Verify with a known-defect reference image."
    )

review_required = bool(suspiciously_empty and models_available)
if review_required:
    quality_score = {
        **quality_score,
        "overall_score": None,
        "grade": "UNVERIFIED",
        "status": "REVIEW_REQUIRED",
        "defect_risk": "UNKNOWN",
        "flagged_reason": (
            "Zero detections from both detectors on a non-trivial image. The score is "
            "deliberately withheld rather than reported as a perfect 100."
        ),
        "formula": "Not computed — zero-detection safety override applied (see
suspicious_zero_detection).",
        "summary": (
            "No components and no defects were detected. A genuinely clean board and a silent "
            "detection failure are indistinguishable here, so the quality score is withheld and "
            "manual review is required."
        ),
    }
# Subtract the XAI sub-window so no wall time is double-counted across buckets.
_postprocess_ms = (time.perf_counter() - _t_post_start) * 1000.0 - _xai_ms
```

```
    _total_ms = (time.perf_counter() - _t_total_start) * 1000.0

    # Deterministic content-derived image id (traceable across systems)
    try:
        _image_id = "IMG-" + hashlib.sha1(
            np.ascontiguousarray(img_bgr).tobytes()
        ).hexdigest()[:12]
    except Exception:
        _image_id = f"IMG-{int(datetime.now().timestamp())}"

    try:
        _mv_component = os.path.basename(self.component_model_path)
    except Exception:
        _mv_component = None
    try:
        _mv_defect = os.path.basename(self.defect_model_path)
    except Exception:
        _mv_defect = None

    severity_summary = [
        {
            "defect_id": d["id"],
            "class": d["class"],
            "level": d["severity"].get("level"),
            "score": d["severity"].get("score"),
            "color": d["severity"].get("color"),
            "explanation": d["severity"].get("explanation"),
        }
        for d in detected_defects
    ]

    timing_ms = {
        "preprocess_ms": round(_preprocess_ms, 4),
        "component_inference_ms": round(_component_inference_ms, 4),
        "defect_inference_ms": round(_defect_inference_ms, 4),
        "xai_ms": round(_xai_ms, 4),
        "postprocess_ms": round(_postprocess_ms, 4),
        "total_ms": round(_total_ms, 4),
        "accuracy_mode": {
            "tta_enabled": bool(self.tta_enabled),
            "ensemble_count": int(len(self.defect_models) if hasattr(self, "defect_models") else 0),
            "per_class_conf_overrides_applied": bool(getattr(self, "per_class_defect_conf_overrides",
None)),
        },
    }

    return {
        # --- SmartPCB-XAI unified inspection contract (v1.0) ---
        "contract_version": CONTRACT_VERSION,
        "inspection_status": "SUCCESS",
        "inspection_id": inspection_id,
        "image_id": _image_id,
```

```
        "timestamp": timestamp,
        "image_dimensions": {"width": orig_w, "height": orig_h},
        "components": detected_components,
        "component_count": n_comps,
        "defects": detected_defects,
        "defect_count": n_defects,
        "quality_score": quality_score,
        "xai": xai_data,
        "severity": severity_summary,
        "recommendations": recommendations,
        "pipeline_warnings": pipeline_warnings,
        "suspicious_zero_detection": suspiciously_empty,
        "review_required": review_required,
        "models_available": models_available,
        "model_version": {
            "component": _mv_component,
            "defect": _mv_defect,
            "engine_version": PROJECT_VERSION,
            "inference_device": getattr(self, "device", None),
        },
        # `inference_time_ms` is the contract field; `inference_timing_ms`
        # is retained unchanged for older callers.
        "inference_time_ms": timing_ms["total_ms"],
        "inference_timing_ms": timing_ms,
        # --- transport attachments (not part of the JSON contract) ---
        "annotated_image": annotated_bgr,
        "xai_overlay_image": xai_overlay_bgr,
    }
```

Defect severity scoring:

```
class SeverityEngine:
    """
    SmartPCB-XAI project-defined severity score & quality scoring engine.

    NOTE: The numeric 0–100 formulas in this module are a project-defined
    deterministic construction

    Fully documented for research auditability.
    (SmartPCB-XAI project-defined severity score)
    """

    DEFECT_BASE_WEIGHTS = {
        "short": 90.0,          # Solder/trace bridging causing catastrophic short circuits
        "open": 85.0,           # Trace disconnection causing circuit functional failure
        "open_circuit": 85.0,   # Trace disconnection (model alias)
        "pin-hole": 60.0,       # Via/pad pinhole causing long-term reliability issues
        "missing_hole": 60.0,   # Missing hole / pinhole (model alias)
        "mousebite": 55.0,      # Edge erosion reducing dielectric withstand voltage
        "mouse_bite": 55.0,     # Edge erosion (model alias)
        "spur": 45.0,           # Trace spur with potential to bridge under vibration
```

```
        "copper": 35.0,          # Spurious copper flakes/etching residue
        "spurious_copper": 35.0,  # Spurious copper (model alias)
    }

    @classmethod
    def calculate_defect_severity(cls, defect_class: str, confidence: float, bbox_area_norm: float =
0.01, near_component: str = None):
        """
        Calculates individual defect severity score (0 to 100).
        """
        base = cls.DEFECT_BASE_WEIGHTS.get(defect_class.lower(), 50.0)
        crit_mult = cls.COMPONENT_CRITICALITY.get(near_component.lower(), 1.0) if
near_component else 1.0

        # Scale with defect area (larger defects have higher impact)
        size_mult = 1.0 + min(1.0, bbox_area_norm * 10.0)

        raw_score = base * crit_mult * size_mult * (0.7 + 0.3 * confidence)
        score = round(min(100.0, max(5.0, raw_score)), 1)

        if score >= 75.0:
            level = "CRITICAL"
            color = "#dc2626"
        elif score >= 50.0:
            level = "HIGH"
            color = "#ea580c"
        elif score >= 25.0:
            level = "MEDIUM"
            color = "#d97706"
        else:
            level = "LOW"
            color = "#16a34a"

        explanation = (
            f"Defect '{defect_class}' has base weight {base}. "
            f"{f'Proximity to critical component {near_component} (x{crit_mult}). ' if near_component
else ''}"
            f"Confidence {confidence*100:.1f}%. Resulting score: {score}/100 ({level})."
        )

        return {
            "score": score,
            "level": level,
            "color": color,
            "base_weight": base,
            "criticality_multiplier": crit_mult,
            "explanation": explanation
        }
```

REFERENCES

[1] P. Bhattacharya and M. F. Cloutier, "Explainable artificial intelligence for industrial applications," in *Proceedings of the IEEE International Conference on Industrial Technology (ICIT)*, 2022, pp. 1–8.

[2] A. Chattopadhay, A. Sarkar, P. Howlader, and V. N. Balasubramanian, "Grad-CAM++: Generalized gradient-based visual explanations for deep convolutional networks," in *Proceedings of the IEEE Winter Conference on Applications of Computer Vision (WACV)*, 2018, pp. 839–847.

[3] J. Chen, K. Cao, and Y. Guo, "Deep learning for surface defect detection on industrial products: A comprehensive review," *IEEE Transactions on Instrumentation and Measurement*, vol. 70, pp. 1–18, 2021.

[4] J. Chen, S. Kao, H. He, W. Zhuo, S. Wen, C. Lee, and S. Yu, "Run, don't walk: Chasing higher FLOPS for faster neural networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 12021–12031.

[5] M. Everingham, L. Van Gool, C. K. Williams, J. Winn, and A. Zisserman, "The Pascal Visual Object Classes (VOC) Challenge," *International Journal of Computer Vision*, vol. 88, no. 2, pp. 303–338, 2010.

[6] IPC, *IPC-A-610G: Acceptability of Electronic Assemblies*. Bannockburn, IL, USA: IPC—Association Connecting Electronics Industries, 2020.

[7] G. Jocher and J. Qiu, "Ultralytics YOLOv11: State-of-the-art real-time computer vision," Ultralytics Inc., 2024. [Online]. Available: https://github.com/ultralytics/ultralytics

[8] Q. Ling and N. A. M. Isa, "Printed circuit board defect detection methods based on image processing, machine learning and deep learning: A survey," *IEEE Access*, vol. 11, pp. 15921–15944, 2023.

[9] D. Ouyang, S. He, G. Zhang, M. Luo, H. Guo, J. Zhan, and Z. Huang, "Efficient multi-scale attention module with cross-spatial learning," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2023, pp. 1–5.

[10] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788. [Online]. Available: https://doi.org/10.1109/CVPR.2016.91

[11] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 618–626. [Online]. Available: https://doi.org/10.1109/ICCV.2017.74

[12] M. Shakya et al., "FICS-PCB: A multi-modal image dataset for automated printed circuit board visual inspection," *IACR Cryptology ePrint Archive*, vol. 2020, p. 1422, 2020.

[13] R. Solovyev, W. Wang, and T. Gabruseva, "Weighted boxes fusion: Ensembling methods for object detection," *Image and Vision Computing*, vol. 107, p. 104117, 2021.

[14] S. Tang, F. He, X. Huang, and J. Yang, "Online PCB defect detector on a new PCB defect dataset," *arXiv preprint arXiv:1902.06197*, 2019.

[15] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, "Digital twin in industry: State-of-the-art," *IEEE Transactions on Industrial Informatics*, vol. 15, no. 4, pp. 2405–2415, 2019.

[16] Z. Tong, Y. Chen, J. Xu, and R. Yu, "Wise-IoU: Bounding box regression loss with dynamic focusing mechanism," *arXiv preprint arXiv:2301.10051*, 2023.

[17] X. Ye, L. Huang, J. Wang, X. Hu, J. Xu, and C. Zhang, "A lightweight PCB defect detection algorithm based on an improved YOLOv11s," *IEEE Access*, vol. 14, pp. 56412–56426, 2026, doi: 10.1109/ACCESS.2026.3678967.

[18] Z. Zhang, "Improved Adam optimizer for deep neural network training in industrial visual inspection," *IEEE Transactions on Industrial Electronics*, vol. 68, no. 8, pp. 7412–7422, 2021.

MAPPING TO SDG GOALS

The SmartPCB-XAI project aligns with the following United Nations Sustainable Development Goals (SDGs):

1. **SDG 9 – Industry, Innovation and Infrastructure:** Uses deep learning and explainable AI for automated PCB defect detection, supporting industrial innovation, improved quality inspection, and reliable electronics manufacturing.
2. **SDG 12 – Responsible Consumption and Production:** Supports early identification of PCB defects, helping reduce material waste, avoid unnecessary rework, and promote more efficient use of manufacturing resources.
3. **SDG 8 – Decent Work and Economic Growth:** Automates parts of the PCB inspection process, potentially reducing repetitive manual inspection effort and supporting productivity in electronics manufacturing.
4. **SDG 4 – Quality Education:** Provides a practical application of deep learning, computer vision, explainable AI, and digital twin technologies, supporting technical learning and skill development.